

LAB TASK#8

NAME: MASHAAL ISHTIAQ

ROLL NO: 22P-9326

SECTION : BCS-4C

QUESTION #1

```
#include <iostream>
using namespace std;

int Multiply(int a, int b)
{

if (b == 0)
return 0; // as a number multiplied with zero is indeed zero ->will return 0 to the function which called it

else
{

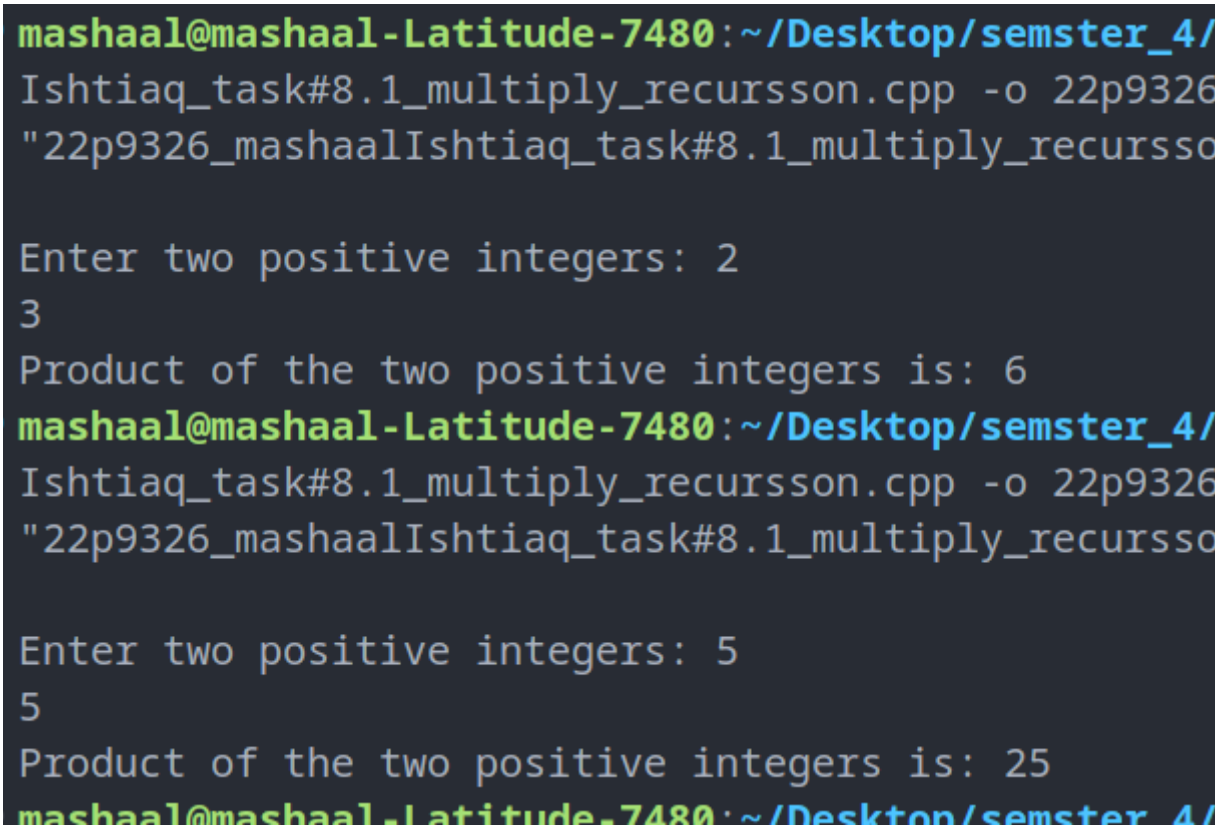
return a + Multiply(a, b - 1);
}
}

int main()
{
int a, b;
cout << "\nEnter two positive integers: ";
cin >> a >> b;

if (a <= 0 || b <= 0)
{
cout << "Invalid input" << endl;
return -1; // to indicate error as a and b should be positive integers
}
int result = Multiply(a, b);

cout << "Product of the two positive integers is: " << result << endl;

return 0;
}
```



QUESTION#2

```
#include <iostream>
using namespace std;

class Node
{

private:
int val;
Node *left;
Node *right;

public:
Node()
{ // default constructor
this->val = 0;
left = right = NULL;
```

```
}

Node(int val)
{ // parameterized constructor
this->val = val;
left = right = NULL;
}

friend class BST;
};
class BST
{

private:
Node *root;

public:
BST()
{
root = NULL;
}
Node *insert(Node *node, int val)
{

if (node == NULL)
{
Node *temp = new Node(val);
if (root == NULL)
{
root = temp;
}
return temp;
}

else
{
if (node->val == val)
{
cout << "Already Exist" << endl;
return node;
}
else if (val < node->val)
node->left = insert(node->left, val);
else
{
node->right = insert(node->right, val);
}
return node;
}
}
void insert_node(int val)
{
```

```
insert(root, val);

cout << "Node has been successfully inserted" << endl;
}

int InOrderTraversal(Node *temp)
{ // here temp is root node

if (temp == NULL)
return 0;
InOrderTraversal(temp->left);

cout << temp->val << "\t";

InOrderTraversal(temp->right);

return 0;
}

int PreOrderTraversal(Node *temp)
{

if (temp == NULL)
return 0;

cout << temp->val << "\t";

InOrderTraversal(temp->left);

InOrderTraversal(temp->right);

return 0;
}

int PostOrderTraversal(Node *temp)
{

if (temp == NULL) // idicated that tree is empty
return 0;

InOrderTraversal(temp->left);

InOrderTraversal(temp->right);

cout << temp->val << "\t";

return 0;
}

int Smallest_value(Node *temp)
{
```

```
if (temp == NULL)
return -1;

Node *curr = temp->left;

while (curr->left != NULL)
{

// as smaller values are on the left side of BST
curr = curr->left;
}
cout << "Smallest value in the tree: " << curr->val << endl;

return 0;
}

int Count_nodes(Node *temp)
{

if (temp == NULL)
return 0;

return Count_nodes(temp->left) + Count_nodes(temp->right) + 1; //+1 as there is root node as well
}

friend int main();
};

int main()
{
BST bst;
int choice, value, result;

while (1)
{

cout << "\n_______Binary Search Tree Operations______\n";
cout << "1. Insert a node\n";
cout << "2. In-order Traversal\n";
cout << "3. Pre-order Traversal\n";
cout << "4. Post-order Traversal\n";
cout << "5. Find Smallest Value\n";
cout << "6. Count Nodes\n";
cout << "7. Exit\n";

cout << "Enter your choice" << endl;
cin >> choice;

switch (choice)
{
```

```
case 1:

cout << "Enter the value" << endl;
cin >> value;

bst.insert_node(value);
break;

case 2:
bst.InOrderTraversal(bst.root);
break;

case 3:
bst.PreOrderTraversal(bst.root);
break;

case 4:
bst.PostOrderTraversal(bst.root);
break;

case 5:
bst.Smallest_value(bst.root);
break;

case 6:

result = bst.Count_nodes(bst.root);

cout << "Total nodes are: " << result << endl;
break;

case 7:
cout << "Exiting..." << endl;
return 0;

default:
cout << "Invalid choice" << endl;
break;
}
}
}
```

```
_mashaalIshtiaq_task#8.2_BST_operations

_______Binary Search Tree Operations______
1. Insert a node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Find Smallest Value
6. Count Nodes
7. Exit
Enter your choice
1
Enter the value
10
Node has been successfully inserted

_______Binary Search Tree Operations______
1. Insert a node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Find Smallest Value
6. Count Nodes
7. Exit
Enter your choice
```

```
Enter your choice
1
Enter the value
8
Node has been successfully inserted

_______Binary Search Tree Operations______
1. Insert a node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Find Smallest Value
6. Count Nodes
7. Exit
Enter your choice
1
Enter the value
11
Node has been successfully inserted

_______Binary Search Tree Operations______
1. Insert a node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Find Smallest Value
6. Count Nodes
7. Exit
```

```
Enter your choice
1
Enter the value
9
Node has been successfully inserted

_______Binary Search Tree Operations______
1. Insert a node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Find Smallest Value
6. Count Nodes
7. Exit
Enter your choice
2
8       9       10      11
_______Binary Search Tree Operations______
1. Insert a node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Find Smallest Value
6. Count Nodes
7. Exit
Enter your choice
3
10      8       9       11
```

```
3. Pre-order Traversal
4. Post-order Traversal
5. Find Smallest Value
6. Count Nodes
7. Exit
Enter your choice
3
10      8       9       11
_______Binary Search Tree Operations______
1. Insert a node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Find Smallest Value
6. Count Nodes
7. Exit
Enter your choice
4
8       9       11      10
_______Binary Search Tree Operations______
1. Insert a node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Find Smallest Value
6. Count Nodes
7. Exit
Enter your choice
```

```
Node has been successfully inserted

_______Binary Search Tree Operations______
1. Insert a node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Find Smallest Value
6. Count Nodes
7. Exit
Enter your choice
6
Total nodes are: 4
```

```
Node has been successfully inserted

_______Binary Search Tree Operations______
1. Insert a node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Find Smallest Value
6. Count Nodes
7. Exit
Enter your choice
5
Smallest value in the tree: 8
```

QUESTION #3

```
#include <iostream>
using namespace std;

class Node
{
public:
int val;
Node *left;
Node *right;

Node(int val)
{
this->val = val;
left = NULL;
right = NULL;
}

friend class BST;
};

class BST
{

public:
Node *root;

BST()
{
root = NULL;
}

Node *Search(Node *root, int val)
{
if (root == NULL)
{
return NULL;
}
else if (root->val == val)
return root;

else if (root->val > val)

return Search(root->left, val);

else
return Search(root->right, val);
```

```
}

void Preorder(Node *root)
{
if (root)
{
cout << root->val << " ";

Preorder(root->left);

Preorder(root->right);
}
}
};

int main()
{
BST s1;

// hard code as insert function is done in 2nd question
Node *root = new Node(10);
root->left = new Node(8);
root->right = new Node(11);
root->left->right = new Node(9);

cout << "\nInput: [" << root->val << "," << root->left->val << "," << root->right->val << "," <<
root->left->right->val << "]" << endl;

int value1 = 8;

Node *result = s1.Search(root, value1);
cout << "\nValue1: " << value1 << endl;
cout << "Output1 : ";
// << "[" << result->val << "]" << endl;
if (result)
{
s1.Preorder(result);
}
else
{
cout << "[]";
}
cout << endl;

// 2nd example

int value2 = 9;

result = s1.Search(root, value2);

cout << "\nValue2: " << value2 << endl;
cout << "Output2 : ";
```

```
// << "[" << result->val << "]" << endl;
if (result)
{
s1.Preorder(result);
}

else
{
cout << "[]";
}
cout << endl;

return 0;
}
```

```
[Running] cd "/home/mashaal/Desktop/semster_4/data_st
22p9326_mashaalIshtiaq_task#8.3_Searching_BST && "/ho

Input: [10,8,11,9]

Value1: 8
Output1 : 8 9

Value2: 9
Output2 : 9

[Done] exited with code=0 in 0.665 seconds
```